



UNIVERSITY OF AZAD JAMMU & KASHMIR
MUZAFFARABAD, AZAD KASHMIR

DIGITAL CLASSROOM MANAGEMENT SYSTEM

C++ Object-Oriented Programming Project Report
SECOND SEMESTER UNIVERSITY PROJECT
Department of Computer Science

PROJECT TITLE	Digital Classroom Management System
PROGRAMMING LANGUAGE	C++
PROGRAMMING PARADIGM	Object-Oriented Programming (OOP)
SEMESTER	2nd Semester
PREPARED FOR	C++ / Object-Oriented Programming Course

C++ / Object-Oriented Programming Course

TABLE OF CONTENTS

1. Introduction
2. Project Overview
3. Problem Statement
4. Objectives
5. System Design and Classes
6. C++ OOP Concepts Used
7. Detailed Code Explanation
8. Runtime Polymorphism and Dynamic Binding
9. Authentication and Role Handling
10. Course Management
11. Program Flow
12. Testing and Expected Output
13. Advantages

Document Scope

This report is based specifically on the supplied console-based C++ source code. It documents the classes, authentication flow, OOP concepts, course data, runtime polymorphism, and program flow implemented in that source.

1. Introduction

The Digital Classroom Management System is a C++ project designed to demonstrate core Object-Oriented Programming concepts through a simple academic management scenario. The implementation models users, students, teachers, and courses using classes, inheritance, virtual functions, constructors, access control, and runtime polymorphism.

This report describes the concepts and functionality demonstrated by the supplied C++ source code.

2. Project Overview

Item	Details
Project	Digital Classroom Management System
Language	C++
Header Files	iostream, string, vector
Main Classes	User, Student, Teacher, Course
Inheritance	Student and Teacher inherit from User
Polymorphism	Virtual showDashboard() overridden by derived classes
Data Structure	vector<Course>
Authentication	Role selection, username and password
Execution	Console-based C++ program

3. Problem Statement

A classroom environment contains different types of users and academic information. A student and a teacher do not perform the same tasks, so the program needs a structure that can represent common user information while allowing role-specific behavior. The project addresses this by creating a base User class and specialized Student and Teacher classes, while a Course class represents academic courses.

4. Objectives

- Demonstrate the fundamental principles of C++ Object-Oriented Programming.
- Create reusable classes for users and courses.
- Use inheritance to represent specialized user roles.
- Use virtual functions and overriding for runtime polymorphism.
- Apply constructors to initialize objects.
- Apply encapsulation through protected/private data members.
- Use vector to store and process multiple Course objects.
- Implement a simple role-based login flow.

5. System Design and Classes

5.1 User Base Class

User is the base class. It stores the common attributes name and role. Its constructor initializes these values. The class also provides a virtual showDashboard() function and a virtual destructor.

```
class User
{
protected:
    string name;
    string role;

public:
    User(string n, string r)
    {
        name = n;
        role = r;
    }

    virtual void showDashboard()
    {
        cout << "\nWelcome to Digital Classroom\n";
    }

    virtual ~User() {}
};
```

5.2 Student Class

Student publicly inherits from User. Its constructor initializes the role as "Student". The class overrides showDashboard() and provides student-specific dashboard options.

```
class Student : public User
{
public:
    Student(string n) : User(n, "Student") {}

    void showDashboard() override
    {
        cout << "\n===== STUDENT DASHBOARD =====\n";
        cout << "Student: " << name << endl;
        cout << "Role: " << role << endl;
        cout << "1. View Courses\n";
        cout << "2. View Attendance\n";
        cout << "3. View Assignments\n";
        cout << "4. View Results\n";
    }
};
```

5.3 Teacher Class

Teacher publicly inherits from User. Its constructor initializes the role as "Teacher". The class overrides showDashboard() and provides teacher-specific management options.

```
class Teacher : public User
{
public:
    Teacher(string n) : User(n, "Teacher") {}

    void showDashboard() override
    {
        cout << "\n===== TEACHER DASHBOARD =====\n";
        cout << "Teacher: " << name << endl;
        cout << "Role: " << role << endl;
        cout << "1. Manage Courses\n";
        cout << "2. Manage Attendance\n";
        cout << "3. Manage Assignments\n";
        cout << "4. Manage Results\n";
    }
};
```

5.4 Course Class

Course is an independent class representing an academic course. Its private members are code, title, and teacher. The constructor initializes these values and display() prints the course information.

```
class Course
{
private:
    string code;
    string title;
    string teacher;

public:
    Course(string c, string t, string tr)
    {
        code = c;
        title = t;
        teacher = tr;
    }

    void display()
    {
        cout << code << " | "
              << title << " | "
              << teacher << endl;
    }
};
```

6. C++ OOP Concepts Used

6.1 Classes and Objects

A class is a blueprint for creating objects. The project defines User, Student, Teacher, and Course classes.

6.2 Encapsulation

Encapsulation keeps data inside a class and controls access. Course uses private members for code, title, and teacher. User uses protected members so derived classes can access common user information.

6.3 Inheritance

Student : public User and Teacher : public User demonstrate inheritance from the User base class.

6.4 Polymorphism

showDashboard() is virtual in User and overridden in Student and Teacher, allowing different behavior through the same interface.

6.5 Abstraction

The User class provides a common interface for dashboard behavior without requiring main() to know each derived implementation.

6.6 Constructors

User, Student, Teacher, and Course each use constructors to initialize objects.

6.7 Function Overriding

Student and Teacher define showDashboard() with the same signature as the virtual function in User. The override keyword confirms the relationship.

6.8 Virtual Functions

The virtual showDashboard() function enables dynamic dispatch. The User destructor is also virtual because derived objects are deleted through a User pointer.

6.9 Access Specifiers

The project uses public, protected, and private access specifiers.

6.10 Collection of Objects

The main function stores multiple Course objects inside vector.

6.11 Dynamic Memory

Student or Teacher objects are created with new, stored in User*, and released using delete currentUser.

7. Detailed Code Explanation

7.1 Header Files

The program includes `iostream` for console input/output, `string` for text data, and `vector` for storing multiple Course objects.

```
#include <iostream>
#include <string>
#include <vector>
```

```
using namespace std;
```

7.2 Base-Class Initialization

The User constructor receives a name and role and assigns them to the corresponding members. This gives every User-derived object common identity information.

7.3 Role-Based Object Creation

After the role, username, and password are checked, the program creates the appropriate derived object through a User pointer.

```
User* currentUser = nullptr;

if (roleChoice == 1 &&
    username == "student" &&
    password == "student123")
{
    currentUser = new Student("Student");
}
else if (roleChoice == 2 &&
        username == "teacher" &&
        password == "teacher123")
{
    currentUser = new Teacher("Teacher");
}
```

7.4 Runtime Polymorphism Call

```
currentUser->showDashboard();
```

The pointer type is `User*`, but the actual object can be `Student` or `Teacher`. Because `showDashboard()` is virtual, the correct overridden function is selected at runtime.

8. Runtime Polymorphism and Dynamic Binding

The program demonstrates dynamic binding by declaring a User pointer, creating a Student or Teacher object based on authentication, assigning the derived object to the base pointer, and calling `currentUser->showDashboard()`. The actual object type determines which dashboard function executes.

Pointer Type	Actual Object	Function Called
User*	Student	Student::showDashboard()
User*	Teacher	Teacher::showDashboard()

9. Authentication and Role Handling

Correct Login Sequence

1. Select Role → 2. Enter Username → 3. Enter Password → 4. Authenticate → 5. Display Dashboard

The main function first asks the user to select a role. After the role is selected, the program asks for a username and password. The credentials are checked according to the selected role.

Role	Username	Password
Student	student	student123
Teacher	teacher	teacher123

If the selected role and credentials do not match, the program displays an invalid-login message and terminates. These credentials are hard-coded for educational demonstration.

10. Course Management

The project creates a vector of Course objects. Five sample courses are added and then displayed using a range-based for loop.

Code	Course	Teacher
CS-201	Object Oriented Programming	Dr. Ahmed
CS-202	Data Structures and Algorithms	Dr. Ali
CS-203	Database Systems	Dr. Hamza
CS-204	Digital Logic Design	Dr. Usman
CS-205	Computer Organization	Dr. Bilal

Relevant Code

```
vector<Course> courses =
{
    Course("CS-201", "Object Oriented Programming", "Dr. Ahmed"),
    Course("CS-202", "Data Structures and Algorithms", "Dr. Ali"),
    Course("CS-203", "Database Systems", "Dr. Hamza"),
    Course("CS-204", "Digital Logic Design", "Dr. Usman"),
    Course("CS-205", "Computer Organization", "Dr. Bilal")
};

for (Course &course : courses)
{
    course.display();
}
```

11. Program Flow

Complete Program Flow

- Program starts in main().
- System title is displayed.
- User selects Student or Teacher role.
- Username is entered.
- Password is entered.
- Credentials are checked according to the selected role.
- A Student or Teacher object is created through a User pointer.
- Runtime polymorphism displays the appropriate dashboard.
- Course objects are created and displayed.
- System status is displayed.
- The dynamically allocated user object is deleted.

- Program terminates.

12. Testing and Expected Output

Test Case	Input / Condition	Expected Result
Student Login	Student / student / student123	Student dashboard displayed
Teacher Login	Teacher / teacher / teacher123	Teacher dashboard displayed
Invalid Role	Any value other than 1 or 2	Invalid role selection message
Invalid Login	Wrong credentials for selected role	Invalid login message
Course Display	Successful login	Five courses displayed
Polymorphism	Student or Teacher object via User*	Correct overridden dashboard runs

13. Advantages

- Clear demonstration of major introductory C++ OOP concepts.
- Reusable User base class reduces duplication between Student and Teacher.
- Runtime polymorphism makes role-specific dashboard behavior possible.
- Course objects provide a clean model for academic information.
- The program is small enough to understand and explain in a second-semester viva.

S,

morphism, vectors, and dynamic memory in one coherent console-based program.